8. Animation

In Android

8. Android Animation

Animation in android apps is:

- the process of creating motion
- and shape change.

https://github.com/idekhail/APA_Animation1

https://www.geeksforgeeks.org/android/animation-in-android-with-example/

The basic ways of animation that we'll look upon in this tutorial are:

1. Fade In Animation
2. Fade Out Animation
3. Cross Fading Animation
4. Blink Animation
5. Zoom In Animation
6. Zoom Out Animation
7. Rotate Animation
8. Move Animation
9. Slide Up Animation
10. Slide Down Animation
11. Bounce Animation
12. Sequential Animation
13. Together Animation

Animation in Android

Animation is the process of adding a motion effect to any view, image, or text.

With the help of an animation, you can add motion or can change the shape of a specific view.

Animation in Android is generally used to give your UI a rich look and feel. The animations are basically of three types as follows:

- **Property Animation**
- **View Animation**
- **Drawable Animation**

View Animation

View Animation can be used to add animation to a specific view to perform tweened animation on views.

Tweened animation calculates animation information such as size, rotation, start point, and endpoint.

These animations are slower and less flexible.

An example of View animation can be used if we want to expand a specific layout in that place we can use View Animation.

The example of View Animation can be seen in Expandable RecyclerView.

Drawable Animation

Drawable Animation is used if you want to animate one image over another.

The simple way to understand is to animate drawable is to load the series of drawable one after another to create an animation.

A simple example of drawable animation can be seen in many apps Splash screen on apps logo animation.

Important functions for working with Animation

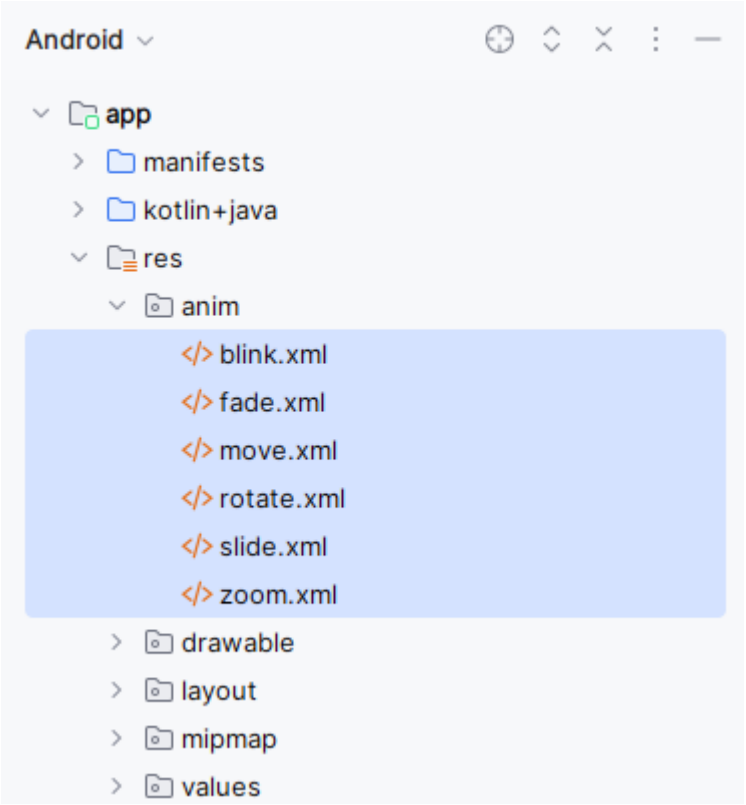
Methods	Description
startAnimation()	This method will start the animation.
clearAnimation()	This method will clear the animation running on a specific view.

Steps of Implementing Android Animation

Now we will see the Simple Example to add animations to ImageView.

Note that we are going to implement this project using **Kotlin** language.

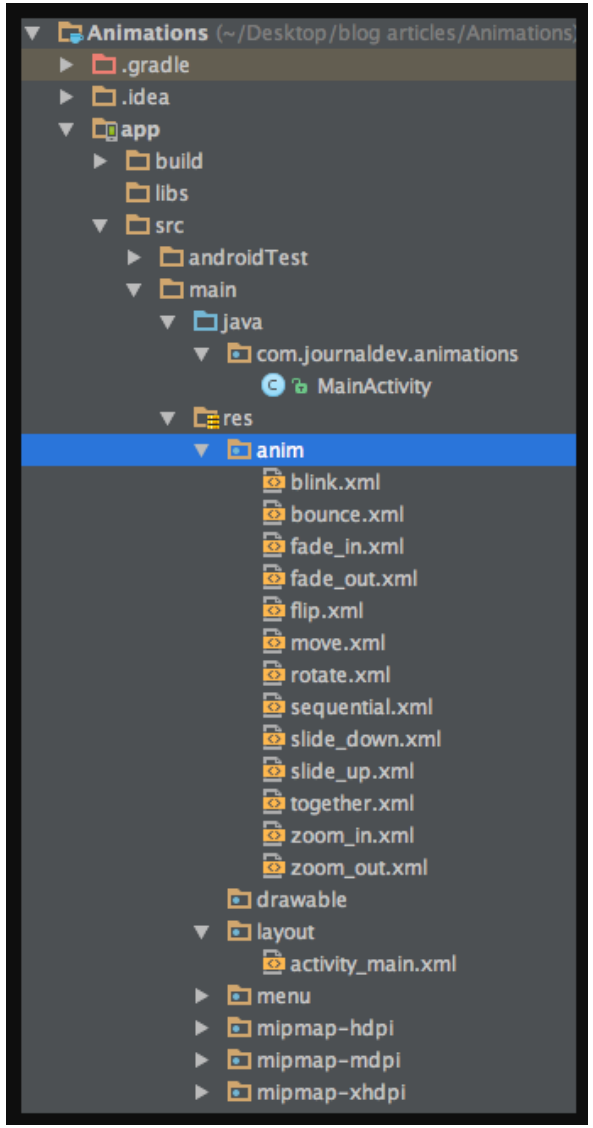
Step 4: Create 6 different types of animation for ImageView



Android Animation Example - 2

- Android Animation is used to give the UI a rich look and feel.
- Animations in android apps can be performed through XML or android code.
- Animation in android apps is the process of creating motion and shape change.
- In this android animation tutorial we'll go with XML codes for adding animations into our application.

Android Animation Project Structure



Android Animation Example XML

```
<?xml version="1.0" encoding="utf-8"?>
<scale xmlns:android="https://schemas.android.com/apk/res/android"
  android:interpolator="@android:anim/accelerate_decelerate_interpolator"
  android:duration="300"
  android:fillAfter="true"
  android:fromXScale="0.0"
  android:fromYScale="0.0"
  android:toXScale="1.0"
  android:toYScale="1.0" />
```

•**android:interpolator** : It is the rate of change in animation. We can define our own interpolators using the time as the constraint. In the above xml code an inbuilt interpolator is assigned

•**android:duration** : Duration of the animation in which the animation should complete. It is 300ms here. This is generally the ideal duration to show the transition on the screen. The start and end of the animation are set using:

- **TRANSFORMATION** : is the transformation that we want to specify. In our case we start with an x and y scale of 0 and end with an x and y scale of 1
- **android:fillAfter** : property specifies whether the view should be visible or hidden at the end of the animation. We've set it visible in the above code. If it sets to false, the element changes to its previous state after the animation
- **android:startOffset** : It is the waiting time before an animation starts. This property is mainly used to perform multiple animations in a sequential manner
- **android:repeatMode** : This is useful when you want the animation to be repeat
- **android:repeatCount** : This defines number of repetitions on animation. If we set this value to infinite then animation will repeat infinite times.

HW4

https://www.digitalocean.com/community/tutorials/android-animation-example#android-animation-example-xml

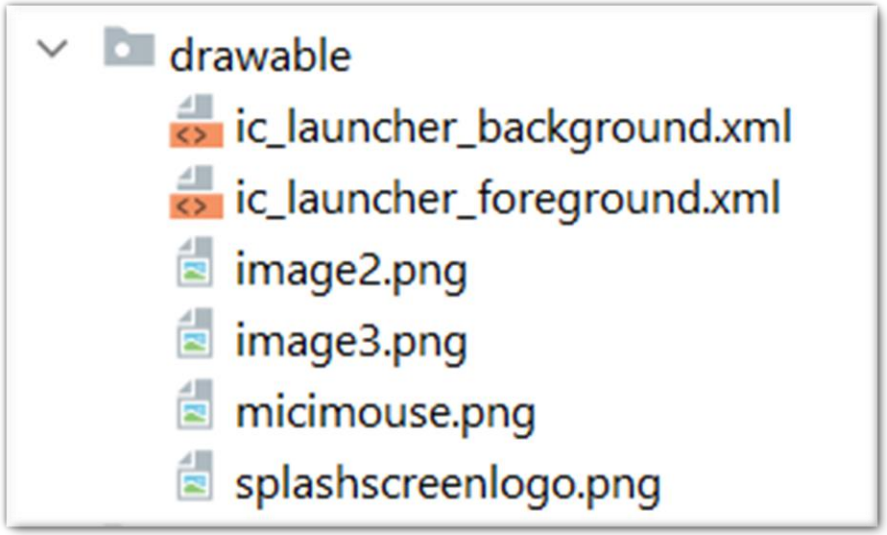
Animation One picture

1. insert **ImageView** with id= mikey
2. Add new image like micimouse.png
3. Link **ImageView** in activity
4. Add Animation class
5. Try these statments

```
<ImageView
    android:id="@+id/mikey"
    android:layout_width="320dp"
    android:layout_height="376dp"
    android:layout_marginTop="76dp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.494"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/ok"
    app:srcCompat="@drawable/micimouse" />
```

```
var mikey = findViewById<ImageView>(R.id.mikey)
mikey.setOnClickListener{ it: View!
    mikey.animate().alpha( value: 0f).duration = 1000
    //mikey.animate().translationXBy( 2000f).duration= 1000
    //mikey.animate().translationXBy(- 2000f).duration= 3000
    //mikey.animate().rotation( 360f).duration= 3000
    //mikey.animate().rotation( -360f).duration= 3000
}
```

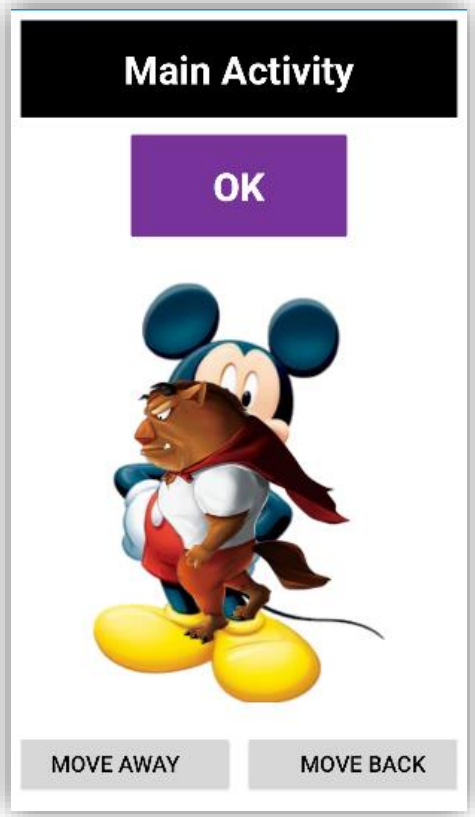
Animation Two pictures (without background)




```
<ImageView
    android:id="@+id/image3"
    android:layout_width="214dp"
    android:layout_height="322dp"
    android:layout_marginTop="76dp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.497"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/ok"
    app:srcCompat="@drawable/image3" />
```

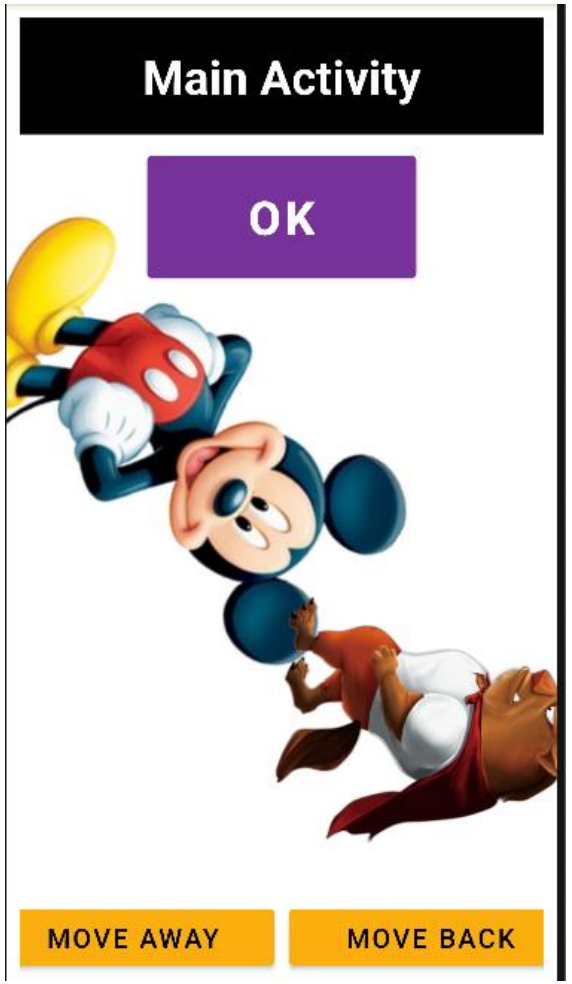
```
<LinearLayout
    android:id="@+id/linearLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center"
    android:layout_margin="10dp"
    android:orientation="horizontal"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/mikey">

    <Button
        android:id="@+id/moveAway"
        android:layout_width="212dp"
        android:layout_height="54dp"
        android:layout_marginBottom="32dp"
        android:layout_margin="5dp"
        android:text="Move Away"
        android:textSize="20dp"
        android:textColor="@color/black" />
    <Button
        android:id="@+id/moveBack"
        android:layout_width="212dp"
        android:layout_height="54dp"
        android:layout_marginBottom="32dp"
        android:text="Move Back"
        android:layout_margin="5dp"
        android:textSize="20dp"
        android:textColor="@color/black"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/mikey" />
</LinearLayout>
```



Animation Two pictures (without background)

```
var image3 = findViewById<ImageView>(R.id.image3)
var moveBack = findViewById<Button>(R.id.moveBack)
var moveAway = findViewById<Button>(R.id.moveAway)
moveAway.setOnClickListener { it: View!
    mikey.animate().rotation( value: 360f)
        .translationXBy( value: -1000f)
        .translationYBy( value: -1000f)
        .duration = 3000

    image3.animate().rotation( value: 360f)
        .translationXBy( value: 1000f)
        .translationYBy( value: 1000f)
        .duration = 3000
}
```

```
moveBack.setOnClickListener { it: View!
    mikey.animate().rotation( value: -360f)
        .translationXBy( value: 1000f)
        .translationYBy( value: 1000f)
        .scaleX( value: 0.5f)
        .scaleY( value: 0.5f)
        .duration = 3000
    image3.animate().rotation( value: -360f)
        .translationXBy( value: -1000f)
        .translationYBy( value: -1000f)
        .scaleX( value: 0.2f)
        .scaleY( value: 0.2f)
        .duration = 3000
}
```